

Algoritmo A*

Análise de Complexidade e Tempo de Execução

Python × JavaScript · Prof. Daniel Bezerra

Arthur Lins da Gama

Arthur Pedrosa Padilha

Pedro Torjal de Medeiros

O problema do caminho mínimo



Entrada

Grade 2D com obstáculos.
Cada célula livre tem custo 1
para o vizinho.



Saída

O menor caminho de A a B,
garantidamente ótimo.



Heurística

Estimativa da distância ao
destino. Sem ela, vira Dijkstra.



Aplicações

Jogos, robôs, GPS, qualquer
domínio com estimativa de
distância.

A ideia: $f(v) = g(v) + h(v)$

f(v) custo total estimado **g(v)** custo real da origem **h(v)** estimativa heurística

01

Inicializar

Custo da origem = 0. Todo o resto começa com custo infinito.
Coloca o ponto de partida na fila de prioridade.

02

Extrair mínimo

Pega o ponto mais promissor da fila (menor custo estimado total).
Se esse ponto já foi processado antes, ignora e pega o próximo.
Se chegamos ao destino, reconstrói e retorna o caminho percorrido.

03

Relaxar vizinhos

Para cada célula vizinha que não é obstáculo:
Calcula o custo de chegar até ela passando por aqui.
Se for melhor do que o caminho já conhecido, atualiza e coloca na fila.

Por que o resultado é ótimo?

Condição 1

Admissível

A estimativa nunca superestima o custo real

Cada passo reduz a distância de Manhattan em no máximo 1, então $h(v)$ nunca ultrapassa o custo real. O A* nunca descarta o caminho ótimo.

Condição 2

Consistente

$h(u) \leq \text{custo}(u,v) + h(v)$ para todo vizinho

Garante que cada célula é visitada no máximo uma vez, evitando reproprocessamento.

Com as duas propriedades, o primeiro caminho encontrado é o mais curto e nenhuma célula é visitada duas vezes.

Passo a passo

 astar.py

```
A*(grade, origem, destino):  
    aberta ← heap por (f, h, ordem)  
    g[origem] ← 0; insere com f=h(origem)  
    enquanto aberta não vazia:  
        v ← remove-mínimo(aberta)  
        se v expandido: continua # lazy  
        marca v como expandido  
        se v = destino: retorna caminho  
        para cada vizinho u livre de v:  
            g' ← g[v] + 1  
            se g' < g[u]:  
                g[u] ← g'; pai[u] ← v  
                insere u com f = g' + h(u)
```


Os três cenários

MELHOR CASO

Grade vazia

Destino na mesma linha

Heurística perfeita: expande só a linha reta.
Tempo dominado pelo overhead do runtime.

$$\Omega(\sqrt{n} \log n)$$

CASO MÉDIO

Obstáculos 25%

30 grades distintas / tamanho

~8% do mapa explorado. PRNG
determinístico garante mapas idênticos em
Python e JS.

$$\approx \Theta(n)$$

PIOR CASO

Labirinto serpentina

Percorre todas as células livres

Labirinto engana a heurística: destino
parece perto, caminho é longo. Visita quase
tudo.

$$O(n \log n)$$

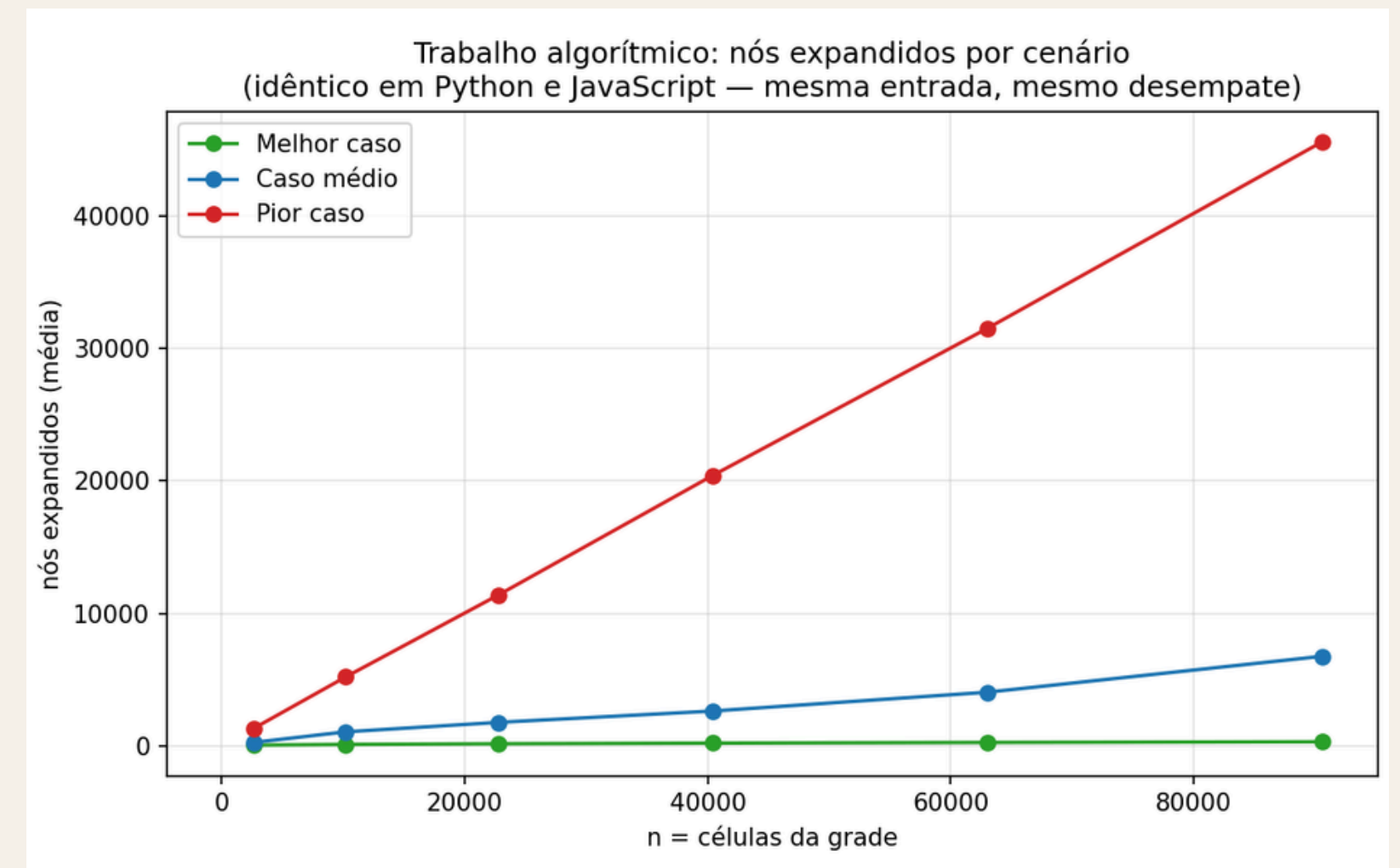
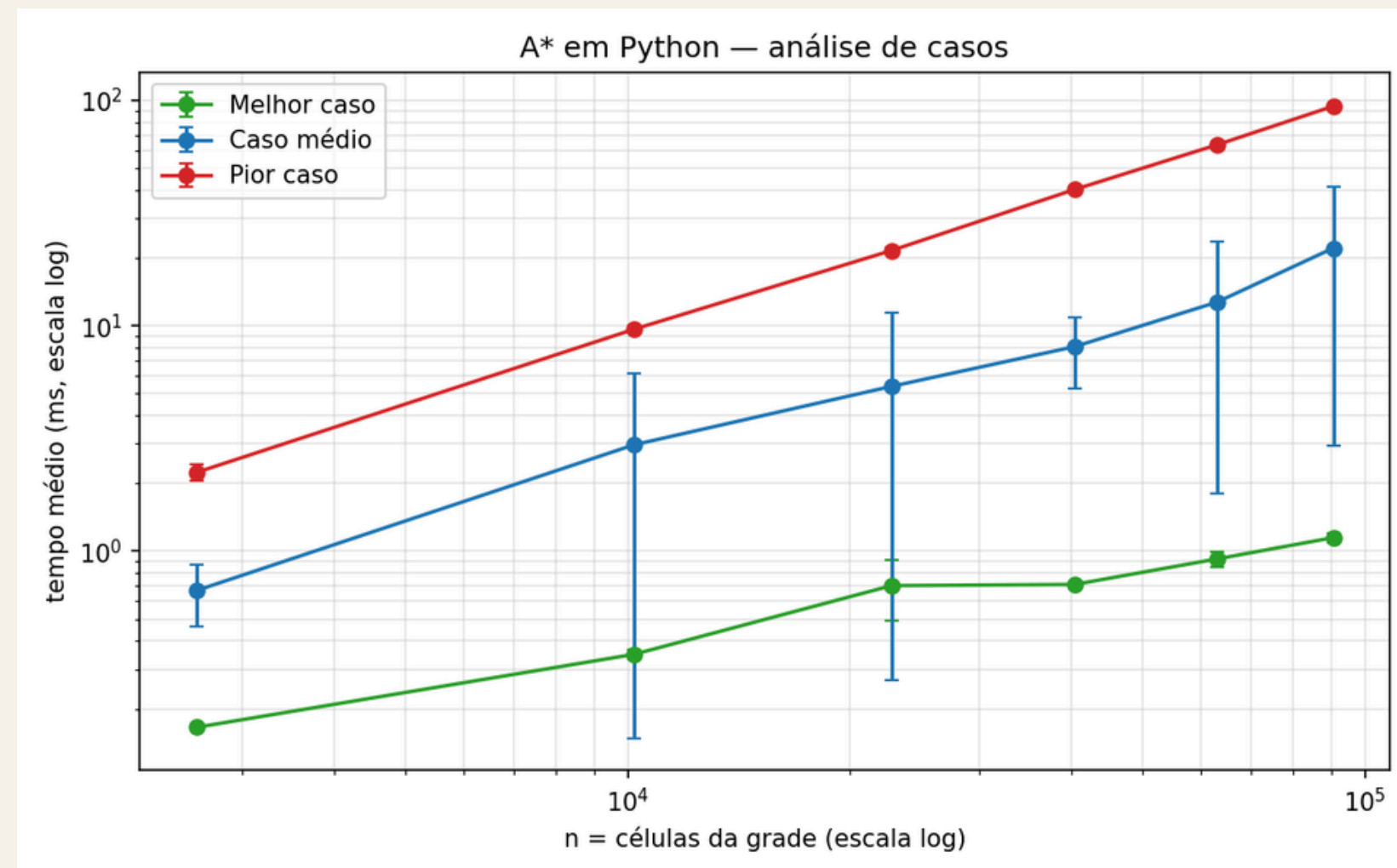
Três regimes e Nós expandidos

Melhor caso: ~1,1 ms @ 301×301

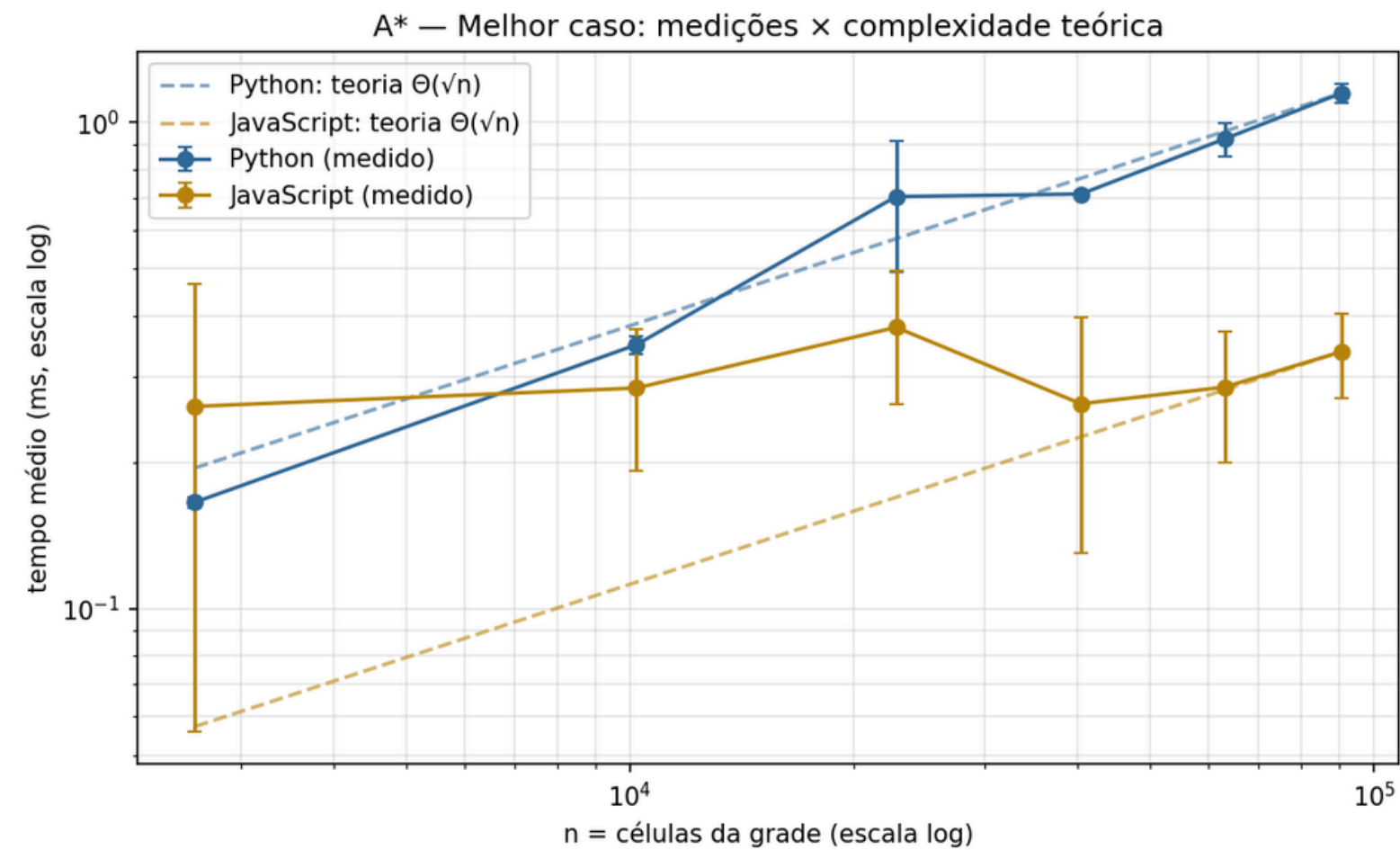
Caso médio: ~22 ms @ 301×301

Pior caso: ~94 ms @ 301×301

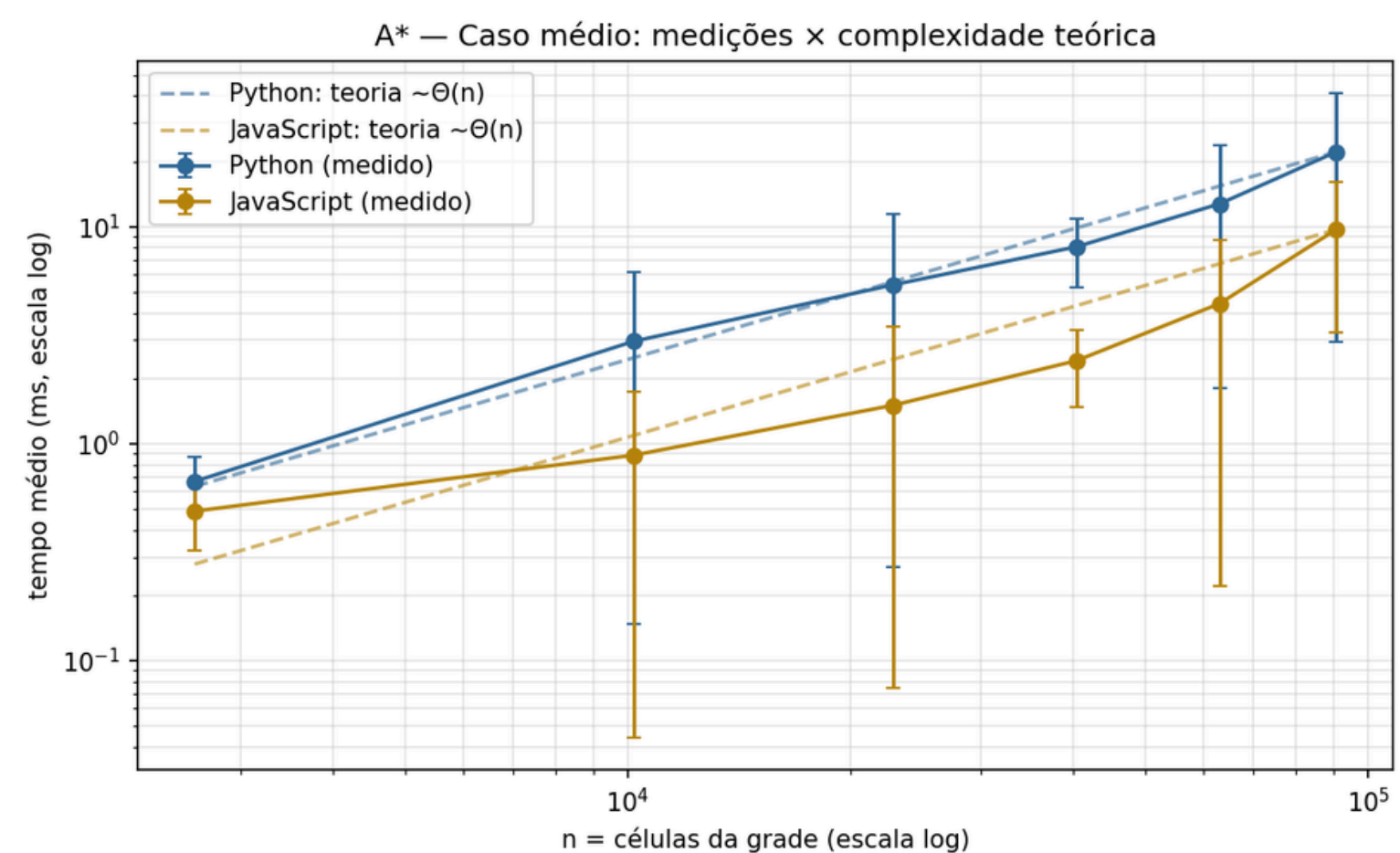
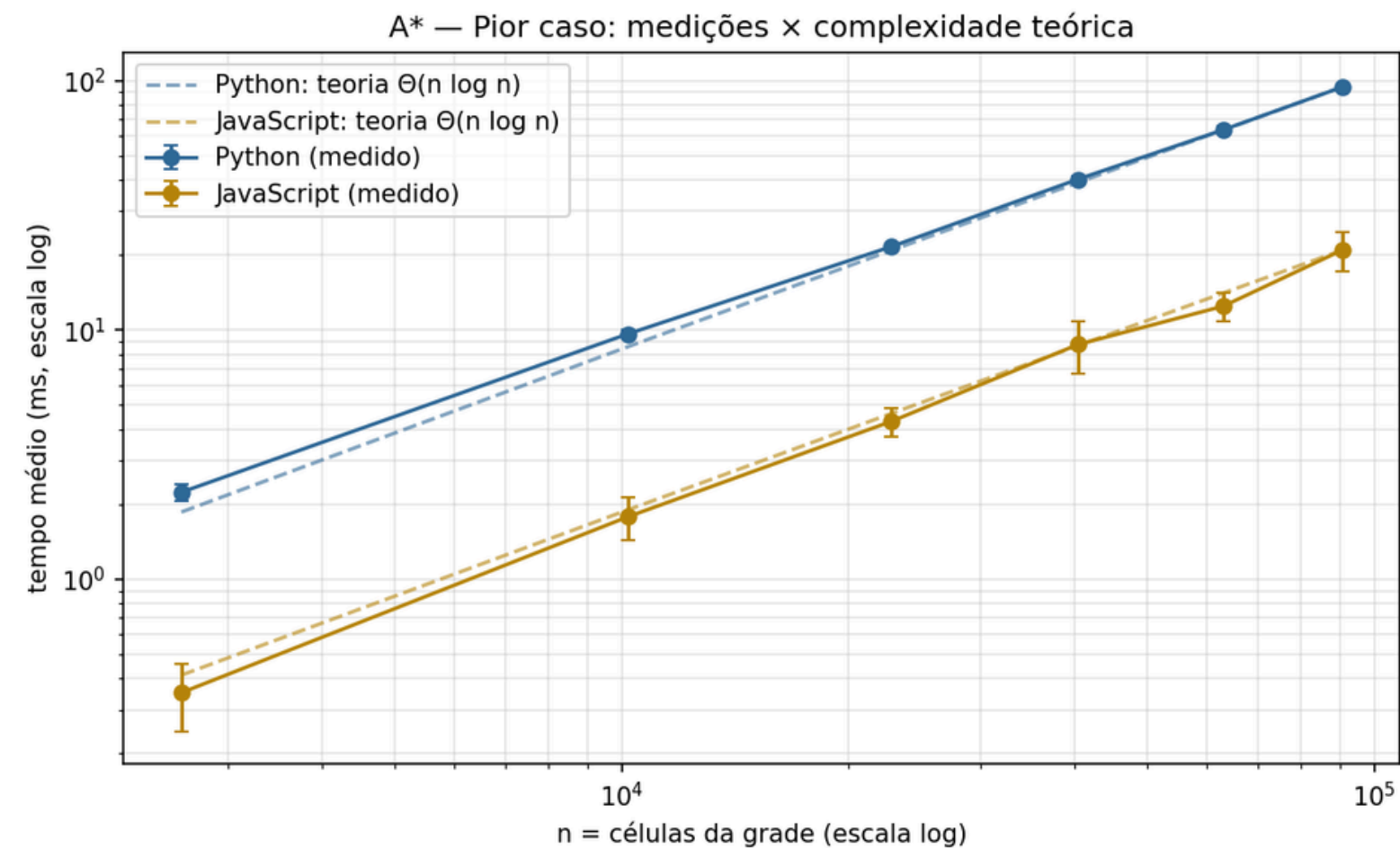
- Python = JavaScript em nós expandidos / Mesmo PRNG e desempate: células visitadas idênticas.
- Trabalho igual, tempo diferente / A diferença de velocidade é custo por operação de cada runtime.
- Pior caso cresce $\frac{1}{2} \cdot n$ / Crescimento linear em nós, $O(n \log n)$ com heap.



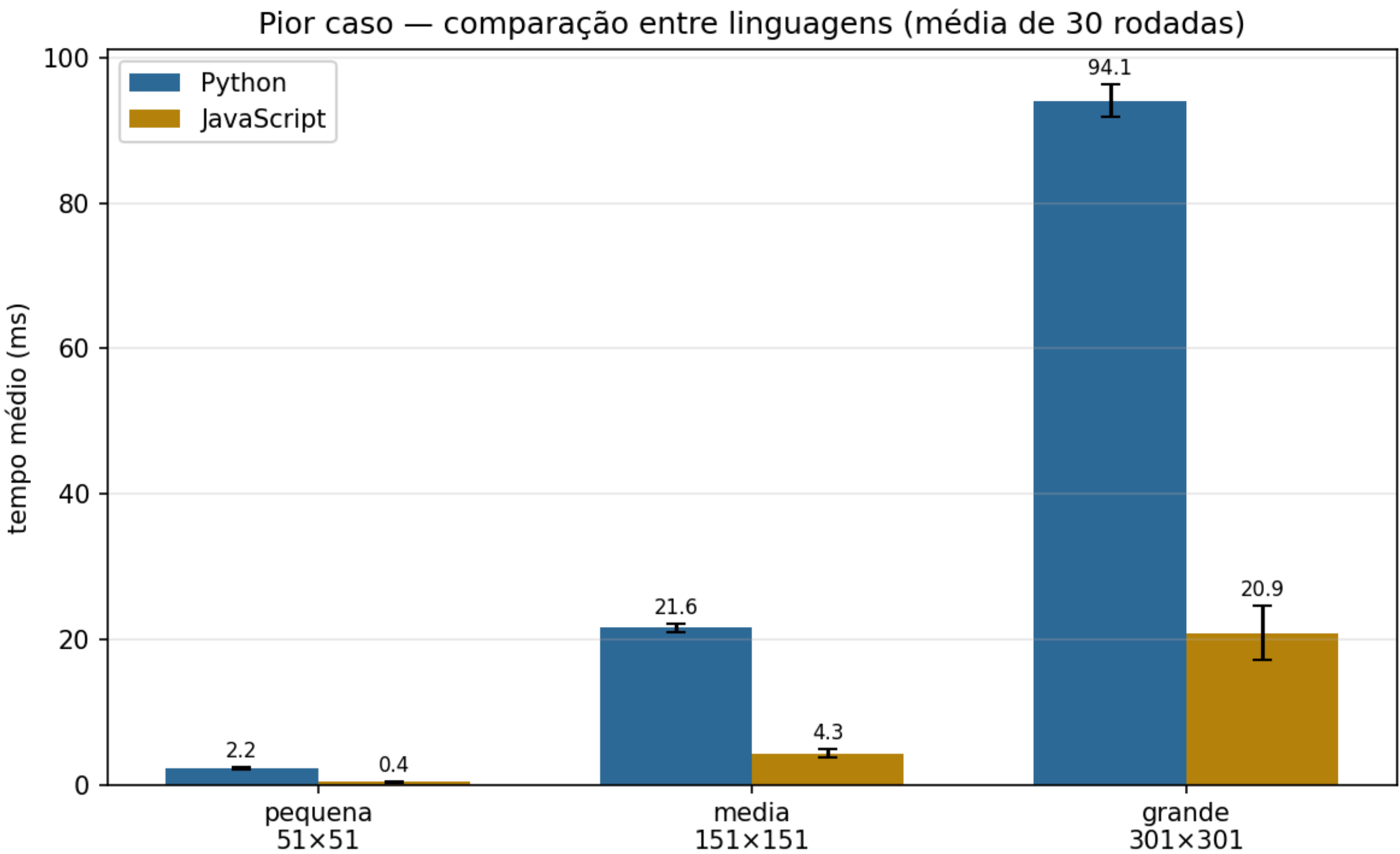
Os três casos e a curva teórica



- Teoria confirmada nos 3 casos: Medições caem sobre a curva teórica em todas as configurações.
- Escalas radicalmente diferentes: Pior caso leva ~85× mais tempo que o melhor no mesmo tamanho.
- Mesma forma de curva: Python e JS crescem juntos, V8 JIT explica a diferença de constante.



Python × JavaScript



Grade 301×301			
Cenário	Python	JS	Razão
Pior caso	94,1 ms	20,9 ms	4,5×
Caso médio	22,1 ms	9,7 ms	2,3×
Melhor caso	1,1 ms	0,34 ms	3,4×

JS é mais rápido por fator constante. A complexidade do algoritmo não muda.

Aplicabilidade e P/NP

Pontos fortes

- Dá pra estimar a distância ao destino sem muito custo
- O grafo não tem muitas conexões por ponto (grade, mapa, rede)
- Jogos · Robôs · GPS · Planejamento de rotas · Quebra-cabeças

Limitações

Memória $O(V)$ → se o mapa for muito grande, o algoritmo pode ficar sem memória. Alternativas como IDA* usam bem menos.

Heurística fraca → se a heurística for ruim, o A* não consegue se orientar e acaba explorando tudo, igual ao Dijkstra.

Mapas dinâmicos → se o mapa muda em tempo real (obstáculos aparecendo), o A* não consegue se adaptar. O D* Lite foi feito pra isso.

Consultas repetidas → se você precisa calcular rotas várias vezes no mesmo mapa (como um GPS), é melhor pré-processar o mapa uma vez e usar esse resultado depois.

P, NP e vizinhos

Caminho mínimo $\in P$

Decidir " $\text{custo} \leq k$?" é polinomial. O A* resolve com eficiência.

Caminho mais longo simples

Contém Hamiltoniano, NP-difícil.

TSP / Multi-agent pathfinding

Ótimo é NP-difícil, cresce exponencialmente.

(n^2-1) -puzzle ótimo

A* acha a solução, mas o espaço é exponencial.

O que o estudo mostrou

Teoria confirmada

Pior caso confirma a curva $n \log n$ em Python e em JS.

Heurística importa

Heurística perfeita: $\sqrt{n}$ nós. Heurística ruim: todos. Ela define o quanto o A* atalha.

JavaScript 4,5x mais rápido no pior caso

V8 JIT compila em tempo real, por isso JS é $\sim 4,5\times$ mais rápido. A curva de crescimento é a mesma.

Complexidade pertence ao algoritmo

Linguagem muda a constante. $O(n \log n)$ é propriedade do algoritmo, não da linguagem.

Agradecemos a Atenção
